Exp No: Date:	SUBSTITUTION TECHNIQUES (CAESAR CIPHER)

AIM:

To implement a Python program to encrypt a message using the Caesar Cipher technique by shifting each alphabet by a specified number of positions.

ALGORITHM:

- Start.
- Get the message and shift value from the user.
- Initialize an empty string encrypted.
- Read each character in the message.
- If the character is an uppercase letter, shift it within A–Z.
- If the character is a lowercase letter, shift it within a–z.
- Keep spaces, numbers, and special characters unchanged.
- Add each converted character to encrypted.
- Display the encrypted message.
- Stop.

PROGRAM:

```
def caesar_cipher(text, shift):
    encrypted = ""

    for char in text:
```

```
        if char.isalpha():
            if char.isupper():
                encrypted += chr((ord(char) - ord('A') + shift) % 26 + ord('A'))
            else:
                encrypted += chr((ord(char) - ord('a') + shift) % 26 + ord('a'))
        else:
            encrypted += char

    return encrypted

message = input("Enter the message: ")
shift = int(input("Enter shift value (0-25): "))

encrypted_message = caesar_cipher(message, shift)
print("Encrypted Message:", encrypted_message)
```

RESULT:

The Caesar Cipher encryption program was successfully executed, and the plaintext was encrypted into ciphertext using the given key.

Exp No: Date:	SUBSTITUTION TECHNIQUES (MONOALPHABETIC CIPHER)

AIM:

To implement a Python program to encrypt a given plaintext using the Monoalphabetic Cipher technique, where each letter of the alphabet is substituted by another unique letter.

ALGORITHM:

- Start.
- Get the message and 26-letter substitution key from the user.
- Check whether the key contains exactly 26 letters.
- Convert the key to uppercase.
- Read each character in the message.
- Find the position of each alphabet in the original alphabet.
- Replace it with the corresponding letter from the substitution key.
- Preserve the case of letters and keep spaces and special characters unchanged.
- Display the encrypted message.
- Stop.

PROGRAM:

```
def monoalphabetic_cipher(text, key):
    alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    cipher = ""
```

```
    key = key.upper()

    for char in text:
        if char.isalpha():
            if char.isupper():
                index = alphabet.index(char)
                cipher += key[index]
            else:
                index = alphabet.index(char.upper())
                cipher += key[index].lower()
        else:
            cipher += char

    return cipher


message = input("Enter the message: ")

print("Enter a 26-letter substitution key")
key = input().strip().upper()
```

```
if (len(key) != 26 or
        not key.isalpha() or
        len(set(key)) != 26 or
        not all(ch in "ABCDEFGHIJKLMNOPQRSTUVWXYZ" for ch in key)):
    print("Invalid key! Enter 26 unique alphabet letters.")
else:
    encrypted = monoalphabetic_cipher(message, key)
    print("Encrypted Message:", encrypted)
```

RESULT:

The Monoalphabetic Cipher encryption program was successfully executed, and the given plaintext was encrypted into ciphertext using the user provided substitution key.

Exp No: Date:	SUBSTITUTION TECHNIQUES (PLAYFAIR CIPHER)

AIM:

To implement a Python program to encrypt plaintext using the Playfair Cipher technique with a 5×5 key matrix and encryption rules.

ALGORITHM:

- Start.
- Get the key and plaintext from the user.
- Convert the key and plaintext to uppercase.
- Remove spaces and special characters from the key and plaintext. Replace the letter J with I.
- Create a 5×5 key matrix using the key and remaining alphabet letters without repetition.
- Divide the plaintext into pairs of letters.
- Insert X between repeated letters in a pair and add X if the final pair contains a single letter.
- Find the positions of both letters in the key matrix.
- If both letters are in the same row, replace them with the letters to their right.
- If both letters are in the same column, replace them with the letters below them.
- Otherwise, replace each letter with the letter in the same row and the other letter's column.
- Display the encrypted text.
- Stop.

PROGRAM:

```
key = input("Enter key: ").upper()

text = input("Enter plaintext: ").upper()


key = ''.join(ch for ch in key if ch in

"ABCDEFGHIJKLMNOPQRSTUVWXYZ").replace("J", "I")

text = ''.join(ch for ch in text if ch in

"ABCDEFGHIJKLMNOPQRSTUVWXYZ").replace("J", "I")


letters = []


for ch in key + "ABCDEFGHIKLMNOPQRSTUVWXYZ":

    if ch not in letters:

        letters.append(ch)


matrix = [letters[i:i + 5] for i in range(0, 25, 5)]


print("\nKey Matrix:")

for row in matrix:

    print(*row)


pairs = []
```

```
i = 0

while i < len(text):
    a = text[i]

    if i + 1 >= len(text):
        b = "X"
        i += 1
    elif text[i] == text[i + 1]:
        b = "X"
        i += 1
    else:
        b = text[i + 1]
        i += 2

    pairs.append((a, b))



def pos(ch):
    for r in range(5):
        for c in range(5):
            if matrix[r][c] == ch:
```

```
            return r, c


cipher = ""
for a, b in pairs:
    r1, c1 = pos(a)
    r2, c2 = pos(b)


    if r1 == r2:
        cipher += matrix[r1][(c1 + 1) % 5]
        cipher += matrix[r2][(c2 + 1) % 5]
    elif c1 == c2:
        cipher += matrix[(r1 + 1) % 5][c1]
        cipher += matrix[(r2 + 1) % 5][c2]
    else:
        cipher += matrix[r1][c2]
        cipher += matrix[r2][c1]


print("Encrypted Text:", cipher)
```

RESULT:

The Playfair Cipher encryption program was successfully executed, and the given plaintext was encrypted into ciphertext using the generated 5×5 key matrix derived from the keyword.

Exp No: Date:	SUBSTITUTION TECHNIQUES (POLYALPHABETIC CIPHER)

AIM:

To implement a Python program to encrypt a given plaintext using the Polyalphabetic Cipher technique, in which multiple substitution alphabets are used with a repeating keyword to produce ciphertext.

ALGORITHM:

- Start.
- Get the plaintext and keyword from the user.
- Convert the plaintext and keyword to uppercase.
- Remove non-alphabetic characters from the plaintext.
- Check that the keyword is not empty and contains only English alphabet
- letters.
- Repeat the keyword until it matches the plaintext length.
- Encrypt each letter using the corresponding keyword letter.
- Apply modulo 26 to obtain the encrypted letter.
- Combine the encrypted letters to form the ciphertext.
- Display the encrypted text.
- Stop.

PROGRAM:

```
def encrypt(plaintext, key):
```

```
    alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"

    plaintext = ''.join(
        ch for ch in plaintext.upper() if ch in alphabet
    )
    key = key.upper()

    if not key or any(ch not in alphabet for ch in key):
        return None

    cipher = ""

    for i in range(len(plaintext)):
        p = ord(plaintext[i]) - ord('A')
        k = ord(key[i % len(key)]) - ord('A')
        c = (p + k) % 26
        cipher += chr(c + ord('A'))

    return cipher


plaintext = input("Enter the plain text: ")
```

```
key = input("Enter the keyword: ").strip()


encrypted = encrypt(plaintext, key)


if encrypted is None:
    print("Invalid key! Enter a non-empty alphabetic keyword.")
else:
    print("Encrypted text (Polyalphabetic Cipher):", encrypted)
```

RESULT:

The Polyalphabetic Cipher encryption program was successfully executed, and the given plaintext was encrypted into ciphertext.

Exp No: Date:	SUBSTITUTION TECHNIQUES (HILL CIPHER)

AIM:

To implement a Python program to encrypt plaintext using the Hill Cipher technique with a key matrix and matrix multiplication.

ALGORITHM:

- Start.
- Get the matrix size and key matrix from the user.
- Get the plaintext and convert it to uppercase.
- Remove spaces and non-alphabetic characters from the plaintext.
- Add the letter X until the plaintext length is divisible by the matrix size.
- Divide the plaintext into blocks.
- Convert each letter into its corresponding numerical value.
- Multiply each block by the key matrix.
- Apply modulo 26 and convert the results into letters.
- Convert the numerical values back into letters.
- Display the encrypted text.
- Stop.

PROGRAM:

```
n = int(input("Enter matrix size: "))
```

```
if n <= 0:

    print("Invalid matrix size!")
    raise SystemExit

print("Enter key matrix:")

key = []
valid = True

for i in range(n):
    row = list(map(int, input().split()))
    if len(row) != n:
        valid = False
        break
    key.append(row)

if not valid:
    print("Invalid matrix! Enter exactly", n, "values in each row.")
    raise SystemExit

text = ''.join(
```

```
    ch for ch in input("Enter plaintext: ").upper()
    if ch in "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
)

if not text:
    print("Invalid plaintext!")
    raise SystemExit

while len(text) % n != 0:
    text += "X"

cipher = ""

for i in range(0, len(text), n):
    block = [ord(ch) - ord('A') for ch in text[i:i + n]]

    for r in range(n):
        total = 0
        for c in range(n):
            total += key[r][c] * block[c]
        cipher += chr((total % 26) + ord('A'))
```

```
print("Encrypted text:", cipher)
```

RESULT:

The Hill Cipher encryption program was successfully executed, and the given plaintext was encrypted into ciphertext using the key matrix and modular arithmetic.

Exp No: Date:	DATA ENCRYPTION STANDARD

AIM:

To implement the Data Encryption Standard (DES) algorithm for encrypting plaintext into ciphertext using a symmetric key encryption technique.

ALGORITHM:

- Start the program.
- Install the required PyCryptodome library using the command python -m
- pip install pycryptodome in the terminal.
- Import the required modules for DES encryption, padding, random key
- generation, and Base64 encoding.
- Generate a 64-bit DES key.
- Get the plaintext message from the user.
- Initialize the DES cipher using the generated key and ECB mode with
- PKCS5 padding (equivalent to PKCS7 padding for DES).
- Encrypt the plaintext using the DES algorithm.
- Encode the encrypted ciphertext into Base64 format.
- Display the encrypted message.
- Decrypt the ciphertext using the same DES key.
- Display the decrypted message.
- Stop the program.

PROGRAM:

```
from Crypto.Cipher import DES
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad
import base64

try:
    key = get_random_bytes(8)
    print("DES Key generated successfully.")

    message = input("Enter the message to encrypt: ")

    cipher = DES.new(key, DES.MODE_ECB)
    padded_message = pad(message.encode(), DES.block_size)
    encrypted_bytes = cipher.encrypt(padded_message)

    encrypted_message = base64.b64encode(
        encrypted_bytes
    ).decode()
    print("Encrypted Message (Base64):", encrypted_message)

    decipher = DES.new(key, DES.MODE_ECB)
```

```
    decrypted_bytes = unpad(
        decipher.decrypt(base64.b64decode(encrypted_message)),
        DES.block_size
    )
    decrypted_message = decrypted_bytes.decode()
    print("Decrypted Message:", decrypted_message)
except Exception as e:
    print("Error:", e)
```

RESULT:

The given plaintext is successfully encrypted into ciphertext and decrypted back to the original message using the Data Encryption Standard (DES) algorithm.

Exp No: Date:	ADVANCED ENCRYPTION STANDARD

AIM:

To implement the Advanced Encryption Standard (AES) algorithm to securely encrypt data using a symmetric key encryption technique.

ALGORITHM:

- Start the program.
- Install the required PyCryptodome library using the command python -m
- pip install pycryptodome in the terminal.
- Import the required modules for AES encryption, key generation,
- padding, and Base64 encoding.
- Generate a 128-bit secret key.
- Display the generated secret key in Base64 format.
- Get the plaintext message from the user.
- Initialize the AES cipher using the generated secret key in ECB mode.
- Apply padding to the plaintext and encrypt it using AES.
- Encode the ciphertext into Base64 format and display the encrypted text.
- Decrypt the ciphertext using the same secret key.
- Remove the padding and display the decrypted text.
- Stop the program.

PROGRAM:

```
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad
import base64

try:
    secret_key = get_random_bytes(16)

    print("Generated Secret Key (Base64 encoded):",
          base64.b64encode(secret_key).decode())

    plain_text = input("Enter text to encrypt: ")

    cipher = AES.new(secret_key, AES.MODE_ECB)
    padded_text = pad(plain_text.encode(), AES.block_size)
    encrypted_bytes = cipher.encrypt(padded_text)

    encrypted_text = base64.b64encode(
        encrypted_bytes
    ).decode()
```

```
    print("Encrypted Text:", encrypted_text)


    decipher = AES.new(secret_key, AES.MODE_ECB)
    decrypted_bytes = unpad(
        decipher.decrypt(base64.b64decode(encrypted_text)),
        AES.block_size
    )

    decrypted_text = decrypted_bytes.decode()
    print("Decrypted Text:", decrypted_text)

except Exception as e:
    print("An error occurred:", e)
```

RESULT:

The given plaintext is successfully encrypted into ciphertext and decrypted back to the original text using the Advanced Encryption Standard (AES) algorithm.

Exp No: Date:	DIFFIE-HELLMAN KEY EXCHANGE

AIM:

To implement a Python program for Diffie-Hellman key exchange, where two parties securely establish a common shared secret key over an insecure channel.

ALGORITHM:

- Start the program.
- Choose a prime number p and a primitive root g of p, which are made
- public.
- Alice chooses a private key a and Bob chooses a private key b.
- Alice computes her public key A=g^a mod p and Bob computes B=g^b
- mod p.
- Exchange the public keys A and B over the insecure channel.
- Alice computes the shared secret S=B^a mod p and Bob computes S=A^b
- mod p.
- Both obtain the same shared secret key S, which is used for secure
- communication.
- Stop the program.

PROGRAM:

```
def power(base, exponent, modulus):
    result = 1
    base = base % modulus

    while exponent > 0:
        if exponent % 2 == 1:
            result = (result * base) % modulus

        exponent = exponent // 2
        base = (base * base) % modulus

    return result

# Public prime and primitive root
p = 23
g = 5

# Private keys
a = 6
b = 15
```

```
# Public keys
A = power(g, a, p)
B = power(g, b, p)

print("Public prime p:", p)
print("Primitive root g:", g)
print("Alice's public key A:", A)
print("Bob's public key B:", B)

# Shared secret keys
secret_alice = power(B, a, p)
secret_bob = power(A, b, p)

print("Shared secret key (Alice):", secret_alice)
print("Shared secret key (Bob):", secret_bob)

if secret_alice == secret_bob:
    print("Key exchange successful!")
else:
    print("Key exchange failed!")
```

RESULT:

Thus, the Diffie-Hellman key exchange program was executed successfully. Both Alice and Bob obtained the same shared secret key, confirming the correctness of the key exchange process.

Exp No: Date:	DIGITAL SIGNATURE USING RSA ALGORITHM

AIM:

To implement the Digital Signature algorithm using RSA in Python for signing a message with a private key and verifying the signature using a public key.

ALGORITHM:

- Start the program.
- Choose two prime numbers p and q.
- Calculate n = p times q.
- Calculate Euler's totient function, $\phi(n)=(p-1)(q-1)$.
- Choose a public exponent e such that gcd$(e,\phi(n))=1$).
- Calculate the private exponent d such that $d \times e \equiv 1 \pmod{\phi(n)}$.
- The public key is (e,n), and the private key is (d,n).
- Convert the message into a numerical value.
- Generate the digital signature using the private key: $S=Md \bmod n$.
- Verify the signature using the public key: $M'=Se \bmod n$.
- Compare the recovered message with the original message and display
- whether the signature is valid.
- Stop the program.

PROGRAM:

p = 3

```
q = 11

n = p * q

phi = (p - 1) * (q - 1)

e = 3

d = pow(e, -1, phi)

print("Public Key  :", (e, n))
print("Private Key :", (d, n))

message = 4

signature = pow(message, d, n)
print("Original Message :", message)
print("Digital Signature:", signature)

verified_message = pow(signature, e, n)
print("Verified Message :", verified_message)
```

```
if verified_message == message:
    print("Digital Signature is valid")
else:
    print("Digital Signature is invalid")
```

RESULT:

The RSA digital signature was successfully generated using the private key and verified using the public key.

Exp No: Date:	HASHING ALGORITHM: SHA-256

AIM:

To implement a Python program that generates hash values for a given input string using built-in cryptographic algorithms (SHA-256) and a simplified custom hash function.

ALGORITHM:

- Start the program.
- Define the input string to be hashed.
- Use hashlib to create a hash instance for SHA-256.
- Convert the input string into bytes using UTF-8 encoding.
- Generate the hash using hexdigest() and convert to hexadecimal string.
- Implement a simple custom hash by arithmetic on character values.
- Display the hash outputs from library-based and custom methods.
- Stop the program.

PROGRAM:

```
import hashlib

def hash_with_library(input_str, algorithm):
    digest = hashlib.new(algorithm, input_str.encode('utf-8'))
```

```
    return digest.hexdigest()

def custom_sha256(input_str):
    hash_val = 7
    for c in input_str:
        hash_val = (hash_val * 31 + ord(c))
    hash_val = hash_val & 0xFFFFFFFF
    return format(hash_val, 'x')

input_str = "Hello, World!"

try:
    sha256_hash = hash_with_library(input_str, "sha256")
    print("SHA-256 Hash (Library):", sha256_hash)

    sha3_hash = hash_with_library(input_str, "sha3_256")
    print("SHA-3-256 Hash (Library):", sha3_hash)

    custom_hash = custom_sha256(input_str)
    print("Custom Hash (Simplified):", custom_hash)

except Exception as e:
```

```
print("Hashing algorithm not found:", e)
```

RESULT:

Thus, program successfully computed the hash values of the input string using SHA-256 and a custom hash function. Each algorithm produced a unique fixed-length hash, confirming the correctness of the hashing process.

Exp No: Date:	MESSAGE AUTHENTICATION USING HMAC

AIM:

To ensure the integrity and authenticity of a message by generating a Hash-based Message Authentication Code (HMAC) using a secret key and a cryptographic hash function (like SHA-256).

ALGORITHM:

- Start the program.
- Secret key K, message M, hash function H (e.g., SHA-256).
- If K > block size -> hash it.
- If K < block size -> pad with zeros.
- Create paddings:
- K_ipad = K XOR ipad (0x36)
- K_opad = K XOR opad (0x5C)
- Compute inner hash:
- inner_hash = H(K_ipad || M)
- Compute final HMAC:
- HMAC = H(K_opad || inner_hash)
- Stop the program.

PROGRAM:

```
import hmac
```

```
import hashlib
import secrets
import base64

def generate_secret_key():
    return secrets.token_bytes(32)

def generate_hmac(message, secret_key):
    hmac_obj = hmac.new(
        secret_key, message.encode('utf-8'), hashlib.sha256
    )
    return base64.b64encode(hmac_obj.digest()).decode('utf-8')

def verify_hmac(message, received_hmac, secret_key):
    calculated_hmac = generate_hmac(message, secret_key)
    return hmac.compare_digest(calculated_hmac, received_hmac)

try:
    shared_secret_key = generate_secret_key()

    print("Generated Secret Key (Base64):",
          base64.b64encode(shared_secret_key).decode('utf-8'))
```

```
    original_message = "This is a confidential message."

    generated_hmac = generate_hmac(original_message, shared_secret_key)
    print("Generated HMAC:", generated_hmac)

    received_message = "This is a confidential message."
    received_hmac = generated_hmac

    is_valid = verify_hmac(received_message, received_hmac,
shared_secret_key)
    print("HMAC verification successful:", is_valid)

    tampered_message = "This is a tampered message."
    is_tampered_valid = verify_hmac(tampered_message, received_hmac,
shared_secret_key)
    print("HMAC verification with tampered message:", is_tampered_valid)

    incorrect_hmac = "some_random_incorrect_hmac_value"
    is_incorrect_valid = verify_hmac(original_message, incorrect_hmac,
shared_secret_key)
    print("HMAC verification with incorrect HMAC:", is_incorrect_valid)
```

```
except Exception as e:
    print("Error:", e)
```

RESULT:

Thus, the above program HMAC ensures the message is secure, unaltered, and from a trusted sender.

Exp No: Date:	KERBEROS AUTHENTICATION SYSTEM SIMULATION

AIM:

To simulate Kerberos authentication in Python and display the authentication result.

ALGORITHM:

- Start the program.
- Create a LoginContext for KerberosLogin.
- Call login() to authenticate the user.
- Retrieve the authenticated user using getSubject().
- Perform privileged actions if authentication is successful.
- Call logout() to end the session.
- Display an error message if authentication fails.
- Stop the program.

PROGRAM:

```
# Kerberos Authentication Simulation


class LoginContext:

    def __init__(self, name):

        self.name = name
```

```
    def login(self):
        raise Exception(
            "No LoginModules configured for " + self.name
        )

    def getSubject(self):
        return "Authenticated User"

    def logout(self):
        print("Logged out successfully!")


try:
    loginContext = LoginContext("KerberosLogin")
    loginContext.login()

    subject = loginContext.getSubject()
    print("Authentication successful!")

    print("Performing privileged actions as the authenticated user...")
```

```
    loginContext.logout()

except Exception as e:
    print("Authentication failed:", e)
```

RESULT:

Thus, the Kerberos authentication simulation was executed successfully, and the authentication failure was displayed.

Exp No: Date:	X.509 CERTIFICATE AUTHENTICATION SIMULATION

AIM:

To simulate X.509 certificate authentication in Python using the cryptography library.

ALGORITHM:

- Start the program.
- Generate an RSA private key and create a self-signed X.509
- certificate.
- Extract the public key from the certificate.
- Verify the certificate's digital signature using the issuer's public key.
- Check the certificate's validity period.
- Display the certificate authentication result.
- Stop the program.

PROGRAM:

```
from datetime import datetime, timezone, timedelta

from cryptography import x509

from cryptography.x509.oid import NameOID

from cryptography.hazmat.primitives import hashes

from cryptography.hazmat.primitives.asymmetric import rsa, padding
```

```
# Generate RSA private key
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)

# Create certificate subject and issuer
name = x509.Name([
    x509.NameAttribute(NameOID.COUNTRY_NAME, "IN"),
    x509.NameAttribute(NameOID.ORGANIZATION_NAME, "ACNS Lab"),
    x509.NameAttribute(NameOID.COMMON_NAME, "localhost")
])

# Generate a self-signed X.509 certificate
now = datetime.now(timezone.utc)

certificate = (
    x509.CertificateBuilder()
    .subject_name(name)
    .issuer_name(name)
    .public_key(private_key.public_key())
```

```
    .serial_number(x509.random_serial_number())
    .not_valid_before(now - timedelta(minutes=1))
    .not_valid_after(now + timedelta(days=365))
    .sign(private_key, hashes.SHA256())
)

print("X.509 Certificate Generated Successfully!")

# Display certificate details
print("\nCertificate Details:")
print("Subject:", certificate.subject.rfc4514_string())
print("Issuer:", certificate.issuer.rfc4514_string())
print("Serial Number:", certificate.serial_number)

# Verify the certificate's digital signature
try:
    certificate.public_key().verify(
        certificate.signature,
        certificate.tbs_certificate_bytes,
        padding.PKCS1v15(),
        certificate.signature_hash_algorithm
    )
```

```
    print("\nCertificate Signature Verification Successful!")

except Exception as e:
    print("\nCertificate Signature Verification Failed:", e)

# Check certificate validity period
current_time = datetime.now(timezone.utc)

if (certificate.not_valid_before_utc <= current_time
        <= certificate.not_valid_after_utc):
    print("Certificate is within its validity period.")
    print("X.509 Certificate Authentication Successful!")
else:
    print("Certificate has expired or is not yet valid.")
print("X.509 Certificate Authentication Failed!")
```

RESULT:

Thus, X.509 certificate generation and authentication were simulated successfully in Python by verifying the digital signature and certificate validity period.

Exp No: Date:	SSL/TLS FOR SECURE CLIENT-SERVER COMMUNICATION

AIM:

To simulate a TLS handshake between a client and server and exchange a secure message.

ALGORITHM:

- Start the program.
- Create a TLS server using an SSL certificate.
- Start the server and listen on port 8443.
- Connect the TLS client to the server.
- Perform the TLS handshake and exchange a message.
- Display the received message.
- Close the connections and stop.

PROGRAM:

```
import ssl

import socket

import threading


def server():

    context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
```

```
    context.load_cert_chain("server.crt", "server.key")

    with socket.socket() as s:
        s.bind(("127.0.0.1", 8443))
        s.listen(1)
        print("TLS Server started on port 8443")

        with context.wrap_socket(s, server_side=True) as ss:
            conn, addr = ss.accept()
            print("Client connected:", addr[0])
            conn.sendall(b"Hello from TLS server")
            print("Message sent to client: Hello from TLS server")
            conn.close()

def client():
    context = ssl._create_unverified_context()

    with context.wrap_socket(
        socket.socket(), server_hostname="localhost"
    ) as s:
        s.connect(("127.0.0.1", 8443))
```

```
        print("Message from server:", s.recv(1024).decode())


t = threading.Thread(target=server)
t.start()


import time
time.sleep(1)


client()
t.join()
```

RESULT:

Thus, the TLS handshake is successfully performed between the client and server, and a message is securely exchanged using Python.

Exp No: Date:	PGP FOR SECURE EMAIL COMMUNICATION

AIM:

To simulate email encryption and decryption using Base64 encoding in Python.

ALGORITHM:

- Start the program.
- Define the message, public key and private key.
- Combine the message with the public key.
- Encode the combined message using Base64.
- Decode the encrypted message using Base64.
- Remove the key and display the original message.
- Stop the program.

PROGRAM:

```
import base64


def encrypt(message, publicKey):
    data = message + publicKey
    encrypted = base64.b64encode(data.encode()).decode()
    return encrypted
```

```
def decrypt(encryptedMessage, privateKey):
    decoded = base64.b64decode(encryptedMessage).decode()
    return decoded.replace(privateKey, "")

message = "Hello, this is a secure email message!"
publicKey = "UserPublicKey"
privateKey = "UserPublicKey"

print("Original Message:", message)

encryptedMessage = encrypt(message, publicKey)
print("Encrypted Message:", encryptedMessage)

decryptedMessage = decrypt(encryptedMessage, privateKey)
print("Decrypted Message:", decryptedMessage)
```

RESULT:

Thus, the PGP encryption and decryption process is simulated using Python, and the original message is successfully recovered.

Exp No: Date:	S/MIME FOR SECURE EMAIL COMMUNICATION

AIM:

To implement a simple S/MIME-like secure e-mail communication process in Python using digital certificates, AES encryption, and RSA encryption.

ALGORITHM:

Step 1: Start the program.

Step 2: Generate an RSA key pair for the receiver.

Step 3: Generate a self-signed X.509 certificate containing the receiver's

public key.

Step 4: Read the e-mail message from the sender.

Step 5: Generate an AES key and encrypt the e-mail message using AES

encryption.

Step 6: Encrypt the AES key using the receiver's RSA public key.

Step 7: Send the encrypted message and encrypted AES key.

Step 8: Decrypt the AES key using the receiver's RSA private key.

Step 9: Decrypt the e-mail message using the recovered AES key.

Step 10: Display the original and decrypted messages.

Step 11: Stop the program.

PROGRAM:

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography import x509
from cryptography.x509.oid import NameOID
from datetime import datetime, timezone, timedelta
import os
import base64


# Generate receiver's RSA key pair
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
public_key = private_key.public_key()


# Create a self-signed X.509 certificate
```

```
name = x509.Name([
    x509.NameAttribute(NameOID.COUNTRY_NAME, "IN"),
    x509.NameAttribute(NameOID.ORGANIZATION_NAME, "S/MIME Lab"),
    x509.NameAttribute(NameOID.COMMON_NAME,
"receiver@example.com")
])

now = datetime.now(timezone.utc)

certificate = (
    x509.CertificateBuilder()
    .subject_name(name)
    .issuer_name(name)
    .public_key(public_key)
    .serial_number(x509.random_serial_number())
    .not_valid_before(now - timedelta(minutes=1))
    .not_valid_after(now + timedelta(days=365))
    .sign(private_key, hashes.SHA256())
)

print("Receiver's X.509 Certificate Created.")
```

```
# Sender writes the email
message = input("Enter email message: ")
plaintext = message.encode("utf-8")

# Encrypt email using AES-GCM
aes_key = AESGCM.generate_key(bit_length=256)
aes = AESGCM(aes_key)
nonce = os.urandom(12)
ciphertext = aes.encrypt(nonce, plaintext, None)

# Encrypt AES key using receiver's RSA public key
encrypted_key = public_key.encrypt(
    aes_key,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

print("\n--- S/MIME-like Secure Email ---")
print("Encrypted AES Key:",
```

```
      base64.b64encode(encrypted_key).decode())
print("Nonce:", base64.b64encode(nonce).decode())
print("Encrypted Email:", base64.b64encode(ciphertext).decode())

# Receiver decrypts AES key using private key
decrypted_key = private_key.decrypt(
    encrypted_key,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

# Decrypt email using AES-GCM
receiver_aes = AESGCM(decrypted_key)
decrypted_message = receiver_aes.decrypt(
    nonce, ciphertext, None
).decode("utf-8")

print("\nDecrypted Email:", decrypted_message)
print("Email decryption successful!")
```

```
# Display certificate information
print("Certificate Subject:",
      certificate.subject.rfc4514_string())
```

RESULT:

Thus, a simplified S/MIME-like secure e-mail communication process was implemented in Python using AES-GCM for message encryption, RSA-OAEP for AES key encryption, and an X.509 certificate for receiver identification.

Exp No: Date:	CONFIGURE AND ANALYZE FIREWALL AND IDS/IPS RULES USING PYTHON

AIM:

To implement and analyze firewall rules and an Intrusion Detection and Prevention System (IDS/IPS) using Python to identify and control suspicious network traffic.

ALGORITHM:

- Start the program.
- Define firewall rules to allow or block network traffic based on IP address
- and port number.
- Read the source IP address, destination port, number of failed login
- attempts, and network data.
- Check the traffic against the configured firewall rules.
- Allow the traffic if it satisfies the rules; otherwise, block it.
- Analyze the network data for suspicious patterns, such as repeated failed
- login attempts or known attack indicators.
- Generate an alert when suspicious activity is detected.
- Apply the simulated prevention rule to block traffic identified as
- malicious.
- Display the firewall decision and IDS/IPS alert.
- Stop the program.

PROGRAM:

```
# Firewall configuration
BLOCKED_IPS = ["192.168.1.100", "10.0.0.50"]
ALLOWED_PORTS = [80, 443]

# Function to check firewall rules
def firewall_check(source_ip, port):
    if source_ip in BLOCKED_IPS:
        return False, "Blocked IP address"

    if port not in ALLOWED_PORTS:
        return False, "Port not allowed"

    return True, "Traffic allowed by firewall"

# Function to detect suspicious activity
def ids_check(failed_attempts, payload):
    suspicious_words = ["malware", "attack", "unauthorized"]

    if failed_attempts >= 5:
        return True, "Multiple failed login attempts detected"
```

```
    for word in suspicious_words:
        if word in payload.lower():
            return True, "Suspicious network data detected"

    return False, "No suspicious activity detected"

# Main program
print("=== Firewall and IDS/IPS Simulation ===")

source_ip = input("Enter source IP address: ")
port = int(input("Enter destination port: "))
failed_attempts = int(input("Enter number of failed login attempts: "))
payload = input("Enter network data: ")

# Firewall inspection
allowed, firewall_message = firewall_check(source_ip, port)
print("\nFirewall Status:", firewall_message)

# IDS inspection
alert, ids_message = ids_check(failed_attempts, payload)
print("IDS Alert:", ids_message)
```

```
# IPS decision
if not allowed or alert:
    print("IPS Action: BLOCK TRAFFIC")
else:
    print("IPS Action: ALLOW TRAFFIC")


print("\nInspection completed.")
```

RESULT:

The Python program was implemented successfully to simulate firewall rule configuration and IDS/IPS analysis. It identifies traffic that violates the defined rules, detects basic suspicious activity, and displays whether the traffic should be allowed or blocked.

Exp No: Date:	**Zero Trust Authentication Using Identity Verification and Access Control**

AIM:

To demonstrate Zero Trust Authentication using Python by verifying user identity and granting access to protected resources based on authentication and access-control policies.

ALGORITHM:

- Start the program.
- Store user details and access permissions.
- Get username, password, device, and IP address.
- Verify the username and password.
- Check the registered device.
- Verify the trusted IP address.
- If verification fails, deny access.
- If successful, generate an authentication token.
- Get the requested resource.
- Check the user's access permission and token expiry.
- Grant or deny access.
- Display the result.
- Stop the program.

PROGRAM:

```
import hashlib

import hmac
```

```
import time


# User details
users = {
    "alice": {
        "password": "Alice@123",
        "role": "admin",
        "device": "laptop1"
    },
    "bob": {
        "password": "Bob@123",
        "role": "staff",
        "device": "laptop2"
    },
    "carol": {
        "password": "Carol@123",
        "role": "guest",
        "device": "phone1"
    }
}


# Access permissions
```

```
permissions = {
    "admin": ["payroll", "firewall", "wiki"],
    "staff": ["payroll", "wiki"],
    "guest": ["wiki"]
}

# Authentication
username = input("Enter username: ")
password = input("Enter password: ")
device = input("Enter registered device: ")
ip = input("Enter IP address: ")

if username in users and hmac.compare_digest(
    password, users[username]["password"]
):
    if device == users[username]["device"]:
        if ip.startswith(("10.", "192.168.")):

            token = hashlib.sha256(
                (username + str(time.time())).encode()
            ).hexdigest()
```

```
            expiry = time.time() + 60

            print("\nIdentity verified successfully!")
            print("Authentication token generated.")

            resource = input(
                "Enter resource (payroll/firewall/wiki): "
            ).lower()

            if time.time() < expiry and resource in permissions[
                users[username]["role"]
            ]:
                print("Access GRANTED")
            else:
                print("Access DENIED")

        else:
            print("Access DENIED: Untrusted network")
    else:
        print("Access DENIED: Unregistered device")
else:
    print("Access DENIED: Invalid username or password")
```

```
print("Zero Trust verification completed.")
```

RESULT:

Thus, Zero Trust Authentication was demonstrated successfully using Python. User credentials, registered devices, network addresses, and access permissions were checked before granting access to protected resources.

Exp No: Date:	**Post-Quantum Secure Message Encryption Using CRYSTALS-Kyber**

AIM:

To implement a post-quantum secure message encryption system using CRYSTALS-Kyber (ML-KEM-768) in Python, derive an AES encryption key from the shared secret, encrypt and decrypt a message, and verify the correctness of the decrypted message.

ALGORITHM:

- Start the program.
- Import the required libraries for Kyber key exchange, hashing, and AES
- GCM encryption.
- Generate a public key and a private key using ML-KEM-768.
- Use the public key to generate a ciphertext and a shared secret at the sender's side.
- Recover the shared secret at the receiver's side using the private key and ciphertext.
- Derive a 256-bit AES key from the shared secret using SHA-256.
- Generate a random nonce and encrypt the original message using AES GCM.
- Decrypt the encrypted message using the recovered shared secret and the same nonce.
- Compare the decrypted message with the original message.
- Display the encrypted data and verification result.
- Stop the program.

- Install the required libraries in the terminal:

  pip install pqcrypto cryptography

PROGRAM:

```
import os

import hashlib

from pqcrypto.kem.ml_kem_768 import keygen, encaps, decaps

from cryptography.hazmat.primitives.ciphers.aead import AESGCM


# Step 1: Generate Kyber key pair

public_key, private_key = keygen()

print("Kyber key pair generated.")


# Step 2: Generate shared secret

kem_ciphertext, sender_secret = encaps(public_key)

print("Shared secret generated by sender.")


# Step 3: Recover shared secret

receiver_secret = decaps(private_key, kem_ciphertext)

print("Shared secret recovered by receiver.")


# Step 4: Generate AES keys
```

```
sender_key = hashlib.sha256(sender_secret).digest()
receiver_key = hashlib.sha256(receiver_secret).digest()

# Step 5: Original message
message = b"Secure communication using Kyber"
print("Original message:", message.decode())

# Step 6: Encrypt message
nonce = os.urandom(12)
aes_sender = AESGCM(sender_key)
encrypted_message = aes_sender.encrypt(nonce, message, None)

print("Encrypted message:", encrypted_message.hex())

# Step 7: Decrypt message
aes_receiver = AESGCM(receiver_key)
decrypted_message = aes_receiver.decrypt(
    nonce, encrypted_message, None
)

print("Decrypted message:", decrypted_message.decode())
```

```
# Step 8: Verify result
if sender_secret == receiver_secret and decrypted_message == message:
    print("SUCCESS: Secure message transfer verified.")
else:
    print("FAILED: Verification unsuccessful.")
```

RESULT:

Thus, a post-quantum secure message encryption system was implemented using ML-KEM-768 and AES-GCM in Python. The shared secret was successfully established, and the encrypted message was decrypted and verified against the original message.

Exp No: Date:	**Homomorphic Encryption and Secure Multi-Party Computation - Homomorphic Encryption using TenSEAL**

AIM:

To perform arithmetic operations on encrypted data using the TenSEAL library without decrypting the original values.

ALGORITHM:

- Start the program.
- Import the TenSEAL library.
- Create a BFV encryption context.
- Generate the encryption keys.
- Encrypt two integer values.
- Perform addition on the encrypted values.
- Decrypt the resulting ciphertext.
- Display the result and stop.
- Stop the program.
- Install the required libraries in the terminal:

  pip install tenseal

PROGRAM:

```
import tenseal as ts
```

```
# Create encryption context
context = ts.context(
    ts.SCHEME_TYPE.BFV,
    poly_modulus_degree=8192,
    plain_modulus=1032193
)

# Input values
a = 25
b = 35

# Encrypt values
enc_a = ts.bfv_vector(context, [a])
enc_b = ts.bfv_vector(context, [b])

# Encrypted addition
enc_result = enc_a + enc_b

# Decrypt result
result = enc_result.decrypt()[0]

print("Encrypted addition completed.")
```

```
print("First value:", a)

print("Second value:", b)

print("Decrypted sum:", result)
```

RESULT:

Homomorphic encryption was successfully demonstrated using TenSEAL, and the encrypted values were added to obtain the correct sum after decryption.

Exp No: Date:	**Secure Multi-Party Computation using MPyC**

AIM:

To compute the sum of private inputs from three parties using secure multi-party computation without directly revealing their individual inputs to other parties.

ALGORITHM:

- Start the program.
- Import the MPyC library and initialize the runtime.
- Define a secure integer data type.
- Assign a private input to each of the three parties.
- Secret-share the inputs using MPyC.
- Compute the sum using secure arithmetic.
- Reveal the final result.
- Display the combined sum and stop.
- Stop the program.
- Install the required libraries in the terminal:
    - pip install mpyc

PROGRAM:

```
from mpyc.runtime import mpc


async def main():
```

```
secint = mpc.SecInt(32)
pid = mpc.pid

# Each party owns one private value
values = [100, 200, 300]

a = mpc.input(
    secint(values[0] if pid == 0 else 0),
    senders=0
)

b = mpc.input(
    secint(values[1] if pid == 1 else 0),
    senders=1
)

c = mpc.input(
    secint(values[2] if pid == 2 else 0),
    senders=2
)

# Secure addition
```

```
    total = a + b + c

    # Reveal only the final result
    result = await mpc.output(total)

    if pid == 0:
        print("SMPC computation completed.")
        print("Combined sum:", result)

mpc.run(mpc.start())
mpc.run(main())
mpc.run(mpc.shutdown())
```

Execution command

```
python smpc_demo.py -M 3
```

RESULT:

Secure multi-party computation was demonstrated using MPyC, allowing three parties to calculate a combined sum using secure arithmetic

Exp No: Date:	**Attribute-Based Encryption (ABE) for Secure Access Control**

AIM:

To implement Attribute-Based Encryption (ABE) for secure access control by encrypting data based on specified attributes and allowing only authorized users whose attributes satisfy the access policy to decrypt the data.

ALGORITHM:

- Start the program.
- Import the required cryptographic libraries.
- Define the attributes of users, such as Department, Role, and Security_Level.
- Specify an access policy, such as allowing access only to users belonging to the CSE department with the role of Faculty.
- Generate a secret encryption key and the required cryptographic parameters.
- Encrypt the confidential message according to the access policy.
- Provide the encrypted message to the user requesting access.
- Check whether the user's attributes satisfy the access policy.
- If the attributes satisfy the policy, decrypt and display the original message.
- Otherwise, deny access and display an unauthorized-access message.
- Stop the program.
- Install - python -m pip install pycryptodome

PROGRAM:

```
import os

from Crypto.Cipher import AES


# Access policy

policy = ["DEPT_CSE", "ROLE_FACULTY"]


# Generate encryption key

key = os.urandom(32)


# Original message

message = b"Confidential department data"


# Encrypt message

cipher = AES.new(key, AES.MODE_GCM)

ciphertext, tag = cipher.encrypt_and_digest(message)


print("=== ATTRIBUTE-BASED ACCESS CONTROL ===")

print("Access policy:", policy)

print("Encrypted message:", ciphertext.hex())


# Authorized user
```

```
authorized = ["DEPT_CSE", "ROLE_FACULTY"]

print("\nAuthorized user:")

if all(attr in authorized for attr in policy):
    decipher = AES.new(key, AES.MODE_GCM, nonce=cipher.nonce)
    plaintext = decipher.decrypt_and_verify(ciphertext, tag)
    print("Access granted")
    print("Decrypted message:", plaintext.decode())
else:
    print("Access denied")

# Unauthorized user
unauthorized = ["DEPT_CSE", "ROLE_STUDENT"]

print("\nUnauthorized user:")

if all(attr in unauthorized for attr in policy):
    print("Access granted")
else:
    print("Access denied: attributes do not satisfy policy")
```

RESULT:

Thus, Attribute-Based Encryption using CP-ABE and AES-GCM can provide secure access control by allowing authorized users to decrypt confidential data while preventing unauthorized access.